

Aula 04 — Métodos Não Paramétricos (KNN)

Curso de Aprendizado de Máquina — UFRJ

Leitura recomendada: AME §4.1–4.5; ISLP §3.5 e cap. 7

Objetivos da aula

Ao final desta aula o aluno deve ser capaz de:

- explicar o que distingue um método *não paramétrico* de um paramétrico e por que ele troca viés por variância;
- descrever a estratégia de *bases* (séries ortogonais, *splines*) para flexibilizar a regressão linear;
- definir o estimador de k **vizinhos mais próximos** (KNN) e o papel de k no balanço viés–variância;
- definir o estimador de **Nadaraya–Watson** e reconhecer os principais *kernels* de suavização e o papel da janela h ;
- entender a **regressão polinomial local** como extensão de Nadaraya–Watson e sua vantagem na fronteira;
- enxergar todos esses métodos como *médias locais* / *suavizadores lineares*.

Em sala

Mudança de paradigma em relação às Aulas 01–02: lá fixávamos a forma de r (linear) e estimávamos poucos parâmetros. Aqui deixamos *os dados ditarem a forma*. Frase-guia (von Neumann): “*com quatro parâmetros eu ajusto um elefante.*” Pergunta: “*e se eu não quiser supor forma alguma para $r(\mathbf{x})$?*”

1 O que é um método não paramétrico?

Um método **paramétrico** supõe que $r(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]$ pertence a uma família indexada por um número *fixo e finito* de parâmetros (ex.: regressão linear, $d + 1$ coeficientes). Um método **não paramétrico** não impõe uma forma funcional fixa: a complexidade efetiva do modelo *crece com n* . A suposição é mais fraca — tipicamente apenas que r é *suave*.

Ideia central

Em relação aos paramétricos, métodos não paramétricos **reduzem o viés** (capturam relações arbitrárias) ao custo de **aumentar a variância**. Continuamos no mundo da Aula 01: o segredo é controlar a flexibilidade por um *tuning parameter* (o k do KNN, a janela h do kernel), escolhido por validação cruzada (Aula 03).

Observação 1.1. Há uma ponte importante com a Aula 02: penalizar um modelo muito flexível (Ridge, Lasso) e usar um modelo não paramétrico são duas formas de se posicionar no balanço viés–variância. A Figura “elefante” do [AME] (§3.9) ilustra isso: penalização *aproxima* um modelo flexível de um simples; não paramétricos *partem* do flexível.

2 Estratégia de bases: séries ortogonais e *splines*

A primeira ideia para ganhar flexibilidade sem sair do arcabouço linear é **ampliar o conjunto de atributos** com funções $\phi_1, \dots, \phi_I$ das covariáveis e ajustar

$$g(x) = \sum_{j=1}^I \beta_j \phi_j(x),$$

que é *linear nos* β_j — estimável por mínimos quadrados (Aula 02). Aqui a flexibilidade é controlada por I (número de funções da base).

2.1 Séries ortogonais

Usa-se uma base ortonormal de L^2 (Fourier, polinômios ortogonais, *wavelets*). Aproxima-se r por suas primeiras I componentes na base; I é o *tuning parameter* (mais termos $\Rightarrow$ mais flexível). É a ideia por trás de “regressão polinomial” vista na Aula 01 (a base dos monômios $1, x, x^2, \dots$).

2.2 *Splines*

Splines são funções polinomiais por partes, “coladas” suavemente em pontos chamados **nós** (*knots*) $t_1, \dots, t_p$. Uma base conveniente para *splines* de grau k é

$$f_1(x) = 1, f_2(x) = x, \dots, f_{k+1}(x) = x^k, \quad f_{k+1+j}(x) = (x - t_j)_+^k, \quad j = 1, \dots, p,$$

onde $(u)_+ = \max(u, 0)$. Com isso $I = k + 1 + p$, e novamente estimam-se os coeficientes por mínimos quadrados. Variantes importantes: *B-splines* (base numericamente estável), *natural splines* (lineares fora dos nós extremos, reduzindo a variância na fronteira) e *smoothing splines* (penalizam a curvatura — caso particular de RKHS, §4.6).

Em sala

Mostre graficamente: poucos nós $\rightarrow$ curva rígida (subajuste); muitos nós $\rightarrow$ curva “nervosa” (superajuste). O número/posição dos nós é o que controla a complexidade. Conecte: “isto é a Aula 01 de novo”.

3 k vizinhos mais próximos (KNN)

O KNN é talvez o método não paramétrico mais intuitivo: para prever em $\mathbf{x}$, faça a *média das respostas dos k pontos de treino mais próximos de $\mathbf{x}$* .

Definição 3.1 (Regressão KNN). Seja $\mathcal{N}_{\mathbf{x}}$ o conjunto dos índices das k observações de treino mais próximas de $\mathbf{x}$ (segundo uma distância d , em geral euclidiana): $\mathcal{N}_{\mathbf{x}} = \{i : d(\mathbf{X}_i, \mathbf{x}) \leq d_{\mathbf{x}}^k\}$, onde $d_{\mathbf{x}}^k$ é a distância do k -ésimo vizinho. Então

$$\hat{r}(\mathbf{x}) = \frac{1}{k} \sum_{i \in \mathcal{N}_{\mathbf{x}}} Y_i. \quad (1)$$

O parâmetro k controla a flexibilidade:

- k **pequeno** (ex.: $k = 1$): vizinhança minúscula, segue cada ponto $\Rightarrow$ *viés baixo, variância alta* (curva “nervosa”, superajuste).
- k **grande**: média sobre muitos pontos; quando $k \rightarrow n$, $\hat{r}$ tende à média global (constante) $\Rightarrow$ *viés alto, variância baixa* (subajuste).

Escolhe-se k por validação cruzada.

Atenção

KNN exige uma noção de distância e, portanto, **depende fortemente da escala** das covariáveis: uma variável em unidade grande domina a distância. *Padronize* antes (Aula 07). Além disso, sofre da *maldição da dimensionalidade*: em d alto, “vizinhos” deixam de ser próximos (tema da Aula 05).

4 Nadaraya–Watson (regressão por kernel)

O KNN dá peso $1/k$ aos k vizinhos e 0 aos demais — uma transição abrupta. Nadaraya–Watson suaviza isso: usa uma **média ponderada** de *todas* as observações, com pesos que decaem com a distância.

$$\hat{r}(\mathbf{x}) = \sum_{i=1}^n w_i(\mathbf{x}) Y_i, \quad w_i(\mathbf{x}) = \frac{K(\mathbf{x}, \mathbf{X}_i)}{\sum_{j=1}^n K(\mathbf{x}, \mathbf{X}_j)}, \quad (2)$$

onde K é um **kernel de suavização** que mede a similaridade entre $\mathbf{x}$ e $\mathbf{X}_i$. Note $\sum_i w_i(\mathbf{x}) = 1$. Escolhas comuns (com janela/largura $h > 0$):

Kernel	$K(\mathbf{x}, \mathbf{X}_i)$
Uniforme	$\mathbb{I}\{d(\mathbf{x}, \mathbf{X}_i) \leq h\}$
Gaussiano	$(2\pi h^2)^{-1/2} \exp(-d^2(\mathbf{x}, \mathbf{X}_i)/(2h^2))$
Triangular	$(1 - d(\mathbf{x}, \mathbf{X}_i)/h) \mathbb{I}\{d(\mathbf{x}, \mathbf{X}_i) \leq h\}$
Epanechnikov	$(1 - d^2(\mathbf{x}, \mathbf{X}_i)/h^2) \mathbb{I}\{d(\mathbf{x}, \mathbf{X}_i) \leq h\}$

O kernel uniforme dá peso igual a todos dentro do raio h (e zero fora); os demais ponderam pela proximidade; o gaussiano dá peso positivo a todos. A janela h é o *tuning parameter*: h grande $\Rightarrow$ muita suavização (viés alto, variância baixa); h pequeno $\Rightarrow$ pouca (variância alta, viés baixo).

Observação 4.1 (Na prática, a janela importa mais que o kernel). Empiricamente, a escolha do *kernel* pouco afeta o resultado; o que importa é o *tuning parameter* h (ou k). Foque a validação cruzada nele.

4.1 Nadaraya–Watson como mínimos quadrados ponderados locais

Há uma leitura iluminadora: $\hat{r}(\mathbf{x})$ em (2) é o intercepto $\hat{\beta}_0$ que resolve

$$\hat{\beta}_0 = \arg \min_{\beta_0} \sum_{i=1}^n w_i(\mathbf{x}) (Y_i - \beta_0)^2. \quad (3)$$

Ou seja: *em cada ponto* $\mathbf{x}$, ajustamos uma **constante** por mínimos quadrados, ponderando as observações pela proximidade a $\mathbf{x}$. É um ajuste linear *local*.

Demonstração. Derivando (3) em β_0 e igualando a zero: $-2 \sum_i w_i(\mathbf{x}) (Y_i - \beta_0) = 0 \Rightarrow \beta_0 = \frac{\sum_i w_i(\mathbf{x}) Y_i}{\sum_i w_i(\mathbf{x})} = \sum_i w_i(\mathbf{x}) Y_i$, pois os pesos somam 1. Logo $\hat{\beta}_0$ coincide com (2). $\square$

5 Regressão polinomial local

Se ajustar uma *constante* local já ajuda, por que não ajustar um *polinômio* local? A regressão polinomial local (LOESS/LOWESS) substitui o intercepto por um polinômio de grau p :

$$\hat{\beta}_0, \dots, \hat{\beta}_p = \arg \min_{\beta_0, \dots, \beta_p} \sum_{i=1}^n w_i(\mathbf{x}) \left(Y_i - \beta_0 - \sum_{j=1}^p \beta_j x_i^j \right)^2, \quad (4)$$

e a predição em $\mathbf{x}$ é $\hat{\beta}_0$. Em forma matricial, com $\mathbf{B}$ tendo i -ésima linha $(1, x_i, \dots, x_i^p)$ e $\mathbf{\Omega} = \text{diag}(w_i(\mathbf{x}))$:

$$(\hat{\beta}_0, \dots, \hat{\beta}_p)^\top = (\mathbf{B}^\top \mathbf{\Omega} \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{\Omega} \mathbf{Y}.$$

Para *cada* $\mathbf{x}$ resolve-se um problema diferente (os pesos mudam). Tomando $p = 0$ recupera-se Nadaraya–Watson.

Ideia central

A vantagem do grau $p \geq 1$ é **reduzir o viés na fronteira** do domínio, onde a média local (NW) é assimétrica e “puxa” a estimativa. O custo é mais variância — escolha o grau (e h) com cuidado.

6 Uma visão unificadora: suavizadores lineares

KNN, Nadaraya–Watson, *splines* e regressão local são todos **suavizadores lineares**: a predição é uma combinação linear das respostas,

$$\hat{r}(\mathbf{x}) = \sum_{i=1}^n \ell_i(\mathbf{x}) Y_i,$$

com pesos $\ell_i(\mathbf{x})$ que dependem só das covariáveis (não de $\mathbf{Y}$). Para o KNN, $\ell_i(\mathbf{x}) = \frac{1}{k} \mathbb{1}_{\{i \in \mathcal{N}_x\}}$; para NW, $\ell_i(\mathbf{x}) = w_i(\mathbf{x})$. Essa estrutura comum explica por que todos têm o mesmo dilema (a largura da vizinhança $\leftrightarrow$ complexidade) e por que valem atalhos como o do LOOCV (Aula 03, Prop. do *leverage*).

7 Prática em Python

```

1 from sklearn.neighbors import KNeighborsRegressor
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.pipeline import make_pipeline
4 from sklearn.model_selection import GridSearchCV
5
6 # KNN com k escolhido por validacao cruzada (padronizando dentro do pipeline!)
7 modelo = make_pipeline(StandardScaler(), KNeighborsRegressor())
8 grade = {"kneighborsregressor_n_neighbors": [1, 3, 5, 11, 25, 51]}
9 busca = GridSearchCV(modelo, grade, cv=5, scoring="neg_mean_squared_error")
10 busca.fit(X_tr, y_tr)
11 print("melhor k:", busca.best_params_)
```

Para Nadaraya–Watson / regressão local, `statsmodels (lowess)` ou `KernelReg` de `statsmodels.nonparametric`. O notebook Aula prática 03, 04 e 05 explora k e h em dados artificiais e no `superconductivity.csv`.

Resumo

- Métodos não paramétricos não fixam a forma de r ; reduzem viés e aumentam variância em relação aos paramétricos.
- *Bases* (séries, *splines*): linearizam o problema; flexibilidade via número de termos/nós.
- **KNN (1)**: média dos k vizinhos; k controla viés–variância.
- **Nadaraya–Watson (2)**: média ponderada por kernel; janela h é o que importa; é uma constante ajustada localmente (3).

- **Regressão polinomial local:** polinômio local; reduz viés de fronteira.
- Todos são *suavizadores lineares*; *padronize* as covariáveis e cuidado com a dimensão (Aula 05).

Leitura recomendada. [AME] §4.1–4.5 (séries, *splines*, KNN, Nadaraya–Watson, regressão local); [ISLP] §3.5 (comparação regressão linear $\times$ KNN) e Capítulo 7 (*Moving Beyond Linearity: splines*, suavização, regressão local).